

Appunti Post pausa. Ore 11.20

Gestione dello Stato di Autenticazione e LocalStorage

Il punto centrale di questa sezione riguarda il concetto di stato dell'applicazione in termini di autenticazione: un utente può trovarsi in uno di due stati possibili, loggato o non loggato, e l'applicazione deve essere in grado di riflettere questa distinzione in modo persistente.

Salvare il token in una variabile Angular non è sufficiente. Una variabile di componente esiste soltanto per la durata di vita del componente stesso: se l'utente ricarica la pagina o naviga altrove, quella variabile viene distrutta e il token va perduto, costringendo l'utente a ripetere il login. Il token deve invece essere conservato in un supporto persistente, che sopravviva ai cicli di vita dei componenti e alle navigazioni. Questo supporto è il `localStorage`.

Il `localStorage` è una piccola base di dati locale messa a disposizione dal browser tramite JavaScript. Si tratta concettualmente di una mappa con chiave di tipo stringa e valore di tipo stringa. Poiché accetta esclusivamente stringhe, per salvare un oggetto strutturato come i dati dell'utente è necessario prima serializzarlo tramite `JSON.stringify()`, e per rileggerlo sarà necessario deserializzarlo tramite `JSON.parse()`. Nel codice mostrato, dopo aver estratto e decodificato il payload dal token JWT, i dati dell'utente vengono scritti nel signal `user` tramite `.set()` e contemporaneamente persistiti nel `localStorage` con la chiave `"user"`.

L'interfaccia `User` viene aggiornata per rispecchiare i campi effettivamente presenti nel payload del token, ovvero `firstName`, `lastName` ed `email`, rimuovendo campi non pertinenti come la password, che non deve mai essere trasmessa o conservata in chiaro nel frontend.

Una conseguenza diretta di questo approccio è che, prima di eseguire una nuova richiesta di login, l'applicazione deve verificare se nel `localStorage` è già presente un utente salvato. Se il token è ancora valido, non è necessario ripetere l'autenticazione, e i dati dell'utente possono essere ripristinati direttamente dalla memoria locale senza ulteriori chiamate al server.

Implementazione del Logout e Utilizzo del Token nelle Richieste

Il logout è un'operazione concettualmente semplice: significa annullare lo stato di autenticazione dell'utente rimuovendo le informazioni salvate in precedenza. A livello pratico, il metodo `doLogout()` esegue due operazioni in sequenza. La prima è la rimozione della voce `"user"` dal `localStorage` tramite `localStorage.removeItem("user")`, eliminando così la persistenza dei dati tra una sessione e l'altra. La seconda è il ripristino del signal `user` al proprio stato iniziale tramite `.set()`, riportando i campi `firstName`, `lastName` ed `email` a stringhe vuote. In questo modo l'interfaccia si aggiorna immediatamente riflettendo il nuovo stato non autenticato.

Ciò che deve essere effettivamente salvato nel `localStorage` non è l'oggetto utente, bensì il token JWT grezzo. Quando il client dovrà effettuare una richiesta verso un endpoint protetto, recupererà il token dal `localStorage` tramite `localStorage.getItem("token")` e lo inserirà nell'header `Authorization` con il formato `"Bearer " + token`, dove lo spazio dopo la parola `Bearer` è parte obbligatoria del formato standard.

Questo schema è visibile nel metodo `doTest()`, dove la chiamata GET all'endpoint protetto riceve come secondo argomento un oggetto di configurazione contenente l'header di autorizzazione costruito dinamicamente leggendo il token direttamente dal `localStorage`. Il risultato della chiamata viene poi scritto in un signal `result` che aggiorna il template, mostrando il numero di righe caricate in caso di successo o il messaggio di errore in caso di fallimento. La pagina mostrata a schermo conferma il funzionamento completo del flusso: l'utente risulta loggato con nome, cognome ed email visibili, e il pulsante di test restituisce la dicitura "Loaded 3 rows", a dimostrazione che la richiesta autenticata ha raggiunto correttamente il backend e ottenuto una risposta valida.

Ecco gli appunti riscritti in forma discorsiva e professionale.

Il flusso di autenticazione con JWT in Spring Security

La parte pubblica dell'applicazione è rappresentata esclusivamente dall'endpoint di login. Tutte le altre API sono private e richiedono autenticazione.

Il `JwtService` si occupa della generazione del token, ma occorre chiedersi dove Spring lo richieda effettivamente. Il meccanismo funziona nel seguente modo: la prima volta l'utente effettua il login, riceve il token, lo memorizza localmente e lo reinvia in ogni chiamata successiva. Il client ha quindi l'obbligo di fornire le proprie credenziali ad ogni richiesta, portando con sé il token come prova della propria identità.

Il sistema di filtri

Spring lavora con un sistema di filtri. Nessuna API è esposta direttamente: Spring la nasconde, interponendo sempre dei filtri tra il client e l'endpoint di destinazione. Quando si lavora con Spring Security, si creano filtri di accesso; quello descritto è solo uno dei tanti possibili, poiché potrebbero essercene dozzine. Per questo si parla di `FilterChain`, ovvero catena dei filtri.

All'interno di questo filtro si analizza la request in arrivo e si decide se riconoscere l'utente oppure no: se passare avanti nella catena senza impostare un utente riconosciuto, oppure se riconoscerlo gestendo i suoi header. A tal fine è necessario il `JwtService` per lavorare con il JSON Web Token e preparare gli eventuali dati per l'utente autenticato.

Il metodo `doFilterInternal`

Il metodo `doFilterInternal` rappresenta l'esecuzione vera e propria del filtro. Riceve tre parametri: la request completa così come arriva, la response (inizialmente vuota) e la `FilterChain`, ovvero l'elenco dei filtri già eseguiti e ancora da eseguire.

La request viene presa nella sua interezza, comprensiva di header e corpo, non solo del JSON. Il filtro preleva il valore dell'header `Authorization`. Se tale header è assente oppure non inizia con il prefisso `Bearer` — richiesto per JWT — il filtro non procede all'autenticazione e passa direttamente al filtro successivo tramite `filterChain.doFilter`. Se invece l'header è presente, viene rimosso il prefisso di sette caratteri (`Bearer`) e si estrae il token JWT grezzo. Da esso si ricava lo username tramite `jwtService.extractUsername(jwt)`.

Il `SecurityContextHolder`

Se lo username è stato estratto correttamente e il `SecurityContext` è ancora vuoto, si caricano i dettagli dell'utente tramite `userService.loadUserByUsername(username)` e si verifica che corrispondano al token tramite `jwtService.isValid`. Il `SecurityContext` è il contenitore in cui Spring memorizza le autenticazioni, ovvero l'utente loggato. Non si tratta del contesto della Dependency Injection, ma di un contesto di sicurezza contenente esclusivamente dati di autenticazione.

Se la validazione ha esito positivo, viene creato un oggetto `UsernamePasswordAuthenticationToken` con i dettagli dell'utente e le sue authority, arricchito con i dettagli della request tramite `WebAuthenticationDetailsSource`, e infine registrato nel `SecurityContext`. Dopodiché il filtro passa il controllo al successivo tramite `filterChain.doFilter`. L'utente risulta autenticato solamente per quella singola request: il `SecurityContext` viene poi utilizzato dalla `SecurityConfiguration` per verificare se il chiamante è loggato oppure no.

La classe `SecurityConfig`

La classe `SecurityConfig` è il punto centrale in cui vengono definite tutte le regole di sicurezza dell'applicazione. È annotata con `@Configuration` e `@EnableWebSecurity`, che indicano a Spring di trattarla come sorgente di configurazione e di abilitare il modulo di sicurezza web del framework.

Il metodo principale è `securityFilterChain`, che riceve un `HttpSecurity` e restituisce la `SecurityFilterChain` applicata ad ogni richiesta. La configurazione si articola in quattro scelte fondamentali. La protezione CSRF viene disabilitata perché è pensata per applicazioni con sessioni e cookie, mentre in un'architettura

stateless basata su JWT la sicurezza è garantita dalla firma del token stesso. Le regole di autorizzazione stabiliscono che solo l'endpoint di login è pubblico tramite `permitAll()`, mentre qualsiasi altra richiesta richiede autenticazione tramite `anyRequest().authenticated()`. La gestione delle sessioni viene impostata su `STATELESS`, il che significa che il server non crea né mantiene sessioni HTTP: ogni richiesta è autonoma e deve portare con sé il token. Infine, il filtro `JwtAuthenticationFilter` viene inserito nella catena tramite `addFilterBefore`, posizionandolo prima del filtro standard `UsernamePasswordAuthenticationFilter`, così da verificare il token JWT prima di qualsiasi altro tipo di autenticazione.

Le annotazioni: @Bean, @Component, @Service, @Repository

L'annotazione `@Bean` indica a Spring che un metodo produce un componente gestito dal container. `@Component`, `@Service` e `@Repository` non cambiano il modo in cui Spring crea l'oggetto, ma danno un significato semantico preciso all'interno dell'architettura.

`@Component` è l'annotazione generica: indica a Spring di creare un'istanza della classe e di caricarla nel contesto. È appropriata per `JwtAuthenticationFilter`, che è un pezzo di logica infrastrutturale da iniettare nella catena dei filtri. `@Service` è una specializzazione di `@Component` dedicata alla business logic: è l'annotazione corretta per `JwtService`, il cui compito è estrarre lo username e validare il token. `@Repository` è la specializzazione dedicata allo strato di persistenza e, oltre a creare il Bean, abilita la traduzione automatica delle eccezioni del database in eccezioni specifiche di Spring.

`@Bean` viene utilizzato principalmente in tre scenari: per trasformare in Bean una classe di terze parti che non si può annotare direttamente (come `BCryptPasswordEncoder`), quando la creazione dell'oggetto richiede logica complessa o parametri specifici, e quando si vuole disaccoppiare l'interfaccia dall'implementazione definendola in un file di configurazione centrale.

Lato frontend: interceptor e guard

Le API dell'applicazione si dividono in pubbliche e private. Il filtro è il meccanismo usato per autenticarsi sul server, mentre Spring Security è ciò che protegge le API. Per evitare di dover allegare manualmente il token ad ogni chiamata dal frontend, si utilizza un interceptor Angular, che lo aggiunge automaticamente ad ogni richiesta in uscita. Le guard, invece, sono già state impostate e il frontend prevede di approfondirle insieme agli interceptor.